

ELABS - Software Interface Specifications

This document outlines the customer-specified software interfaces required by the ELABS platform. It details the versions, protocols, message formats, and functional relationships for the database server, real-time messaging gateway, and deployment runtime engine.

1. MySQL Database Server Interface

Metadata:

- (1) Name:** MySQL Database Server
- (2) Mnemonic (Encoding):** INT-DB-MYSQL (UTF-8 / Binary Protocol)
- (3) Specification Number:** SPEC-INT-001
- (4) Version Number:** 8.0
- (5) Source:** Oracle Corporation (MySQL Community Server)

(1) Discussion of Purpose:

MySQL is the core relational storage engine for the ELABS platform. It stores persistent data schemas including user credentials, role maps (RBAC), laboratory timetables, student check-in histories, active occupancy counts, equipment inventories, borrowing logs, and session completion checklists. The Node.js Express server establishes connections via standard relational connection pools (Prisma ORM), while the FastAPI Vision and AI backend services interact with it via SQLAlchemy to query and update attendance and inventory records synchronously.

(2) Definition of the Interface (Message Content and Format):

- Format: SQL queries (DDL, DML, DQL) transmitted over standard TCP connections (default port 3306) utilizing the MySQL Client/Server Protocol.
- Message Content:
 - Inputs (Queries): Parameterized SQL strings (e.g., `SELECT * FROM attendance_records WHERE student_id = ?`).
 - Outputs (Result Sets): Raw row tables, metadata arrays detailing affected rows,

database schema verification errors, or execution status headers (OK/ERR) encoded in UTF-8.

2. Socket.IO Engine Interface

Metadata:

- (1) **Name:** Socket.IO Engine
- (2) **Mnemonic (Encoding):** INT-WS-SOCKETIO (JSON / UTF-8)
- (3) **Specification Number:** SPEC-INT-002
- (4) **Version Number:** 4.7
- (5) **Source:** Socket.IO Open Source Project (MIT License)

(1) Discussion of Purpose:

The Socket.IO engine provides a real-time, low-latency, bidirectional communication layer between the ELABS backend (Express API Gateway) and the frontend applications (Next.js Web Console and Expo Mobile Client). Its primary purpose is the instant propagation of safety events (fire or smoke hazard alerts detected by YOLOv8 vision workers), real-time occupancy counting updates, and barcode checkout completion notifications during QR scan workflows.

(2) Definition of the Interface (Message Content and Format):

- **Format:** WebSocket protocol (ws:// or wss://) packets encapsulated in Socket.IO protocol frames (default port 4000), utilizing JSON payloads.
- **Message Content:**
 - **Event safety:alert:** Real-time safety broadcasts. Format: {"event": "fire_detected", "location": "Machines Laboratory", "timestamp": "2026-06-20T16:44:54Z", "severity": "CRITICAL"}
 - **Event occupancy:update:** Real-time student count syncs. Format: {"labId": "EIE-LAB-01", "activeCount": 18, "timestamp": "2026-06-20T16:44:54Z"}
 - **Event borrow:status:** Sync status updates during technician scanner checkouts. Format: {"transactionId": "TXN-77952", "itemId": "EQUIP-042", "status": "APPROVED", "studentId": "STUD-2023-102"}

3. Docker Engine Interface

Metadata:

- (1) **Name:** Docker Engine
- (2) **Mnemonic (Encoding):** INT-DEP-DOCKER (YAML / CLI Commands)
- (3) **Specification Number:** SPEC-INT-003
- (4) **Version Number:** 24.0+ (Community Edition)
- (5) **Source:** Docker Inc.

(1) Discussion of Purpose:

Docker Engine serves as the system containerization platform to standardize the deployment environment across EIE department server nodes. It ensures that the computer vision and AI microservices (FastAPI + YOLOv8 + OpenCV + Ollama Local LLM), which require specific system runtimes and CUDA library dependencies, run in isolated containers with exact software packages without altering or polluting the host operating system.

(2) Definition of the Interface (Message Content and Format):

- **Format:** Standard YAML structure (docker-compose.yml v3.8) and Docker REST API control requests sent via the host OS named pipe or UNIX daemon socket (/var/run/docker.sock).
- **Message Content:**
 - **Configuration (YAML):** Container configurations defining base execution environments (e.g., python:3.10-slim, node:18-alpine), volume storage bindings (./packages/shared:/app/shared), mapped TCP ports (3000:3000, 4000:4000, 8001:8001), and required GPU capabilities (nvidia runtime block for hardware-accelerated YOLO models).
 - **API Commands:** Docker Daemon commands (e.g., build image, restart container, fetch logs, check container health status) formatted as standard HTTP/REST payloads.